LANCASTER UNIVERSITY

Computing Department

PREview: Tackling the Real Concerns of Requirements Engineering

Pete Sawyer, Ian Sommerville and Stephen Viller

Cooperative Systems Engineering Group

Technical Report : CSEG/5/1996

CSEG, Computing Department, Lancaster University, LANCASTER, LA1 4YR, UK.
Phone: +44-1524-65201 Ext 3799; Fax: +44-1524-593608; E-Mail: julie@comp.lancs.ac.uk

PREview: tackling the real concerns of requirements engineering

Pete Sawyer, Ian Sommerville and Stephen Viller
Computing Department
Lancaster University
Lancaster
UK
LA1 4YR

tel: (44) 1524 593780
fax: (44) 1524 593608
[sawyer, is, viller]@comp.lancs.ac.uk

Abstract

A common complaint from industrial development projects is that systems fail to address the real concerns of the major stakeholders. This results in much rework as missing requirements are identified, or incorrectly stressed requirements are changed at later stages in the life-cycle. A major cause of these problems is that the overriding needs become subsumed in the general welter of requirements information. For example, safety-critical systems enter service which are compromised by secondary requirements such as efficiency or throughput.

These high-level, paramount needs, which are frequently vaguely expressed, represent the true concerns of the major stakeholders. As such they should be carefully elaborated and then used to drive the requirements process in such a way that conflicting requirements are identified early and negotiated away. This paper describes a method called PREview which integrates concerns into a viewpoint-oriented approach. PREview mandates the explicit identification and elaboration of concerns at the commencement of a requirements analysis and orients the rest of the process to the assurance of compliance with those concerns. It provides a light-weight and pragmatic approach to requirements engineering where the need to address concerns permeates every stage of the process.

Keywords: Concerns, Requirements, Viewpoints

1. Introduction

In a recent review of open issues in requirement engineering, Zave [Zave 95] identified the following as an outstanding problem area:

Generating strategies for converting vague goals (e.g. "user friendliness", "security", "reliability") into specific properties or behaviour.

The top-level aims of a project's principal stakeholders frequently come into this category. In such cases, the achievement of the goals is critical to the success of an enterprise. For example, embedded control software for defence, transportation, chemical plant, etc. applications must typically be constrained by safety goals. Similarly, security and availability are the overriding goals of banking applications, while usability is an essential goal of mass-market desktop applications. The difficulty facing requirements engineering is ensuring that these goals:

- Are elaborated into concrete requirements which can be explicitly addressed by subsequent design and verification phases. In addition to "product" requirements on the software, these may also include "process" requirements on the development process.

- Act to constrain the rest of the requirements process so that they are not subverted by conflicting but otherwise valid requirements. Failure to detect and resolve such conflicts must inevitably lead to rework or worse [Hutchings 95].

This paper describes a requirements method called PREview (Process and REquirements viewpoints) which is tailored to explicitly address the conversion of top-level goals into requirements and constraints. In PREview, these goals are termed "concerns" and are integrated with a viewpoint-oriented approach to requirements discovery and analysis.

PREview addresses the early stages of the requirements engineering process where requirements and requirements' sources must be discovered and documented before detailed analysis and system modelling can commence. At this early stage, a viewpoint-oriented approach can aid identification of the stakeholders, helping to ensure "probable completeness" [Mullery 79] as well as providing separation of concerns and an aid to traceability [Finkelstein 96]. The explicit association of concerns with viewpoints makes them permeate the whole requirements process. They act to drive this process by effectively setting an agenda by which viewpoints' requirements are discovered, analysed and documented with constant reference to the need for compliance with, and satisfaction of, the concerns.

The remainder of this paper is structured as follows: Section 2 provides an overview of the requirements process and the role of viewpoints within this. Section 3 describes PREview's process model and the interaction between concerns, viewpoints and requirements within this. An application from the railway domain is used as an exemplar. Section 4 describes PREview's current status, summarises the lessons which we have learned from evaluation of the method and draws final conclusions. These are that PREview is a pragmatic method which is adaptable to industrial requirements processes where it is essential for concerns to be explicitly modelled and addressed.

2. Viewpoints and the requirements process

2.1 The requirements process

The objective of the requirements process is to develop a requirements specification document which defines the system to be procured and which can act as a basis for the system design. There are several possible models of this process which could be derived. PREview adopts a view of the process as shown in figure 1 (after Boehm [Boehm 88, Boehm 94]). We believe that this model is broadly compatible with most defined or de-facto industrial requirements processes.

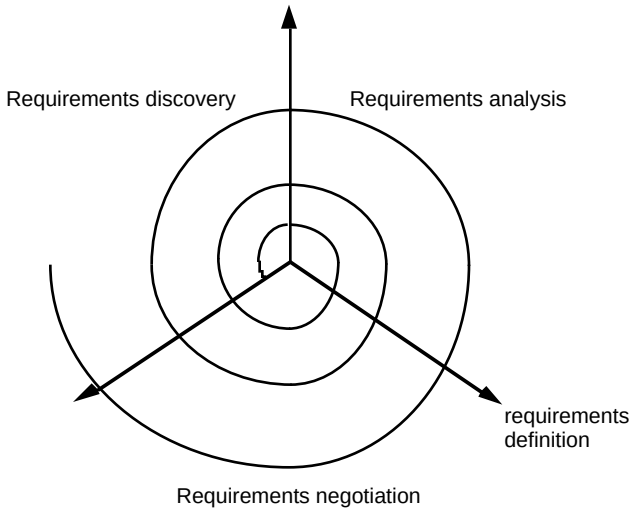

Figure 1 A spiral model of the requirements engineering process

There are three phases shown as segments of the spiral:

1. *Requirements discovery* Given a statement of organisational needs and other inputs, various different sources are consulted to understand the problem and the application domain and to establish their views on what the system requirements should be. We deliberately use the term “discovery” rather than the more common “capture” or “elicitation” to reflect the diverse nature of requirements sources from human users through organisational stakeholders to fundamental underpinning domain phenomena. Requirements are seldom simply available for capture, nor can they be elicited from an inanimate source. They must be actively sought and discovered.

2. *Requirements analysis* The requirements collected during the discovery phase are integrated and analysed. Usually, this will result in the identification of missing requirements, inconsistencies and requirements conflicts.

3. *Requirements negotiation* The system stakeholders consider the identified set of requirements and agree on a sub-set of the requirements for the system. Generally, there are a greater number of desirable requirements than can be implemented so decisions have to be made at this stage about leaving out requirements and modifying requirements to result in a lower-cost system. A result of this phase is that phase 1 must typically be re-entered in order to discover further information and refine existing but incomplete information.

The radial arms of the spiral represent not only increasing cost, but also the generation of information by all three phases. We have labelled this requirements definition. The implication of figure 1 is that requirements engineering is an iterative process where the discovery/analysis/negotiation cycle is iterated around a number of times until the requirements information meets some acceptable level of completeness and consistency within the constraints imposed by cost and the availability of resources. The generated requirements definition information is used as the basis of a requirements specification document.

The challenge with the handling of top-level concerns is that they must first be identified and then translated into concrete requirements and constraints. To be effectively handled within this model, concerns should emerge at the apex of the spiral

and act to checkpoint the requirements information as it emerges from each segment with each iteration.

Figure 2 illustrates an adaptation of the spiral model (which has been tilted to aid clarity) where concerns are identified at the outset and emerge at the apex of the spiral. Thereafter, they radiate out along the of the axes where they act as compliance checks for the requirements emerging from each segment.

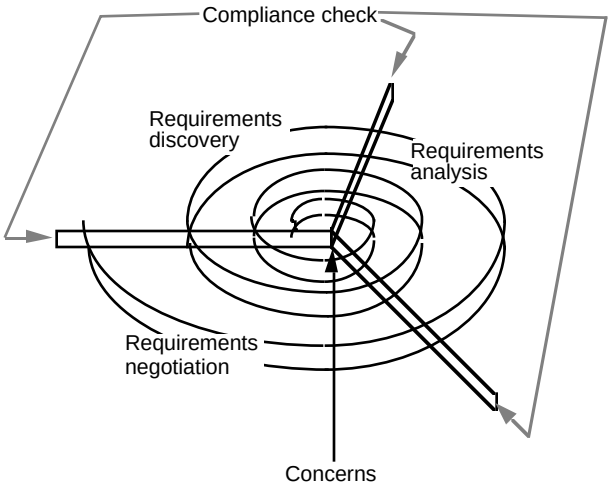

Figure 2 Concerns and the spiral requirements model

The mechanisms by which this is achieved are integrated within PREview's viewpoints model. The three segments (discovery, analysis and negotiation) form the principal phases within the process defined for the application of PREview. These are preceded by the identification and elaboration of concerns which then permeate the process.

Requirements discovery is composed of the identification of the relevant system viewpoints and the requirements arising from viewpoints. Concerns are explicitly associated with each viewpoint (according to their relevance) and are used here to pose questions as the viewpoints' requirements are "discovered" (e.g. when interviewing a stakeholder, interpreting an interface specification to an existing system, etc.). This acts as a means to trap inconsistencies at a very early stage and hence avoid later reworking of non-compliant requirements.

Requirements analysis is concerned with the detection of inconsistencies; both between viewpoints' requirements and, even more importantly, between viewpoint requirements and requirements elaborated from the concerns. Typically this will expose errors and these must be dealt with by the following activity.

Requirements negotiation is where the viewpoints whose requirements conflict with other viewpoints or with concern requirements are re-analysed (with their sources if available) in order to resolve the errors. In many cases this will require reworking of the viewpoints' requirements. In more serious cases, it may reveal the presence of other viewpoints or the incorrect partitioning of the domain viewpoints. This necessitates the reformulation of existing, and the identification of new, viewpoints. It implies a return to requirements discovery.

2.2. Requirements and viewpoints

It has long been recognised that it is good practice to analyse a software or systems engineering problem from the perspectives of the various actors (human or machine) who must interact with the system or who have some stake in the system. The term "viewpoint" is broadly synonymous with perspective. Davis [Davis 93] likens the use of viewpoints in requirements engineering to the practice of using projections on data to build selective but internally cohesive views of the available information.

The principal advantages offered by viewpoints to requirements engineering are:

- By explicitly identifying the viewpoints on a system, the union of the sets of all the viewpoints' requirements is likely to be more complete than if the viewpoints have not been identified. In the latter case, important requirements may be easily overlooked because their sponsors (viewpoints) were never recognised. Of course, completeness cannot be guaranteed but Mullery [Mullery 79] uses the term "probable completeness" to suggest that incompleteness is minimised.

- A separation of concerns if provided which permits the requirements engineer to develop a set of "partial specifications" in isolation from other (possibly competing) viewpoints. This helps to "...master (a problem's) complexity; the smaller problems should be simpler than the large problem. By the same decomposition we hope to factor out subproblems that are already solved and to reuse their existing solutions" [Jackson 96]. Discovery of requirements is explicitly decoupled from analysis of the requirements to detect and resolve errors such as inconsistencies between viewpoints' requirements.

- Traceability is enhanced by the explicit association of requirements with the viewpoints from which they are derived.

Although intuitively simple, viewpoints require methodological support in order to apply them to best advantage. Structured methods, such as JSD [Jackson 83] or the various flavours of object-oriented analysis (e.g. [Rumbaugh 91, Jacobsen 93, Booch 94]) incorporate an implicit notion of viewpoints. These methods suggest that a number of different system models be developed, each of which can be thought of as a different viewpoint on the system. SADT [Ross 77, Schoman 77] and CORE [Mullery 79] go further by explicitly suggesting that different viewpoints should be identified.

Despite these methods' advocacy, viewpoints remain hard to apply in industrial projects. This is due to a number of reasons including: the difficulty in identifying viewpoints (in some methods a viewpoint is vaguely defined, in others a viewpoint is merely considered to be a data sink or source); the problems of managing the information collected by viewpoints; and the problems of integrating viewpoints to identify and resolve inconsistencies.

In recent years several groups have attempted to address these problems. Leite's work [Leite 91] and VORD [Kotonya 96] principally address the problems of viewpoint identification. However, their definitions of viewpoint makes their application problematic in some application domains. For example Leite's method makes it hard to handle viewpoints from inanimate sources and requires the viewpoint information to be encoded in a specialised notation which may be inappropriate at the early stages of a requirements exercise. VORD is oriented towards interactive systems where requirements are mapped to services provided by the system. This is an inconvenient convention for certain classes of embedded systems.

Viewpoints work conducted at Imperial College and Sussex University [Easterbrook 93, Nuseibeh 94, Easterbrook 96] is particularly focused on the problems of detecting and resolving inconsistencies. They show that in order to handle inconsistencies, a requirements method must tolerate inconsistencies until it is appropriate to resolve them. For example, resolution of two inconsistent requirements should be deferred until sufficient global information (knock-on effects to other requirements, relative mutability of the requirements, etc.) is available to inform the necessary trade-off. As with [Leite 91], the work necessitates the encoding of requirements in a particular notation (although a method engineering phase permits some flexibility in the notation chosen), the semantics of which permit the partial automation of the detection of functional inconsistencies.

While non-functional requirements may eventually map down onto functional requirements, this is generally impractical during the early stages of the requirements process. At this stage, non-functional requirements must generally be documented using informal notations. This makes them intractable to automatic reasoning and thus restricts the applicability of approaches oriented towards the partial automation of inconsistency detection.

3. Concerns and viewpoints in PREview

PREview has been developed as part of the REAIMS (Requirements Engineering Adaptation and IMprovement for Safety and dependability) project, an EC-funded project concerned with requirements methods and processes for dependable systems. PREview has been strongly influenced by the experiences of our industrial partners whose priorities were to improve their requirements process. None had made use of viewpoints explicitly but all agreed that the approach had the potential to contribute to improving their processes. However, the nature of their business (the development of dependable embedded systems) made them subject to overriding concerns (e.g. safety) and the need to cope with diverse sources of requirements (e.g. regulatory authorities). Clearly, our work needed to incorporate these within a viewpoints model for such a model to be applicable.

The output from an application of PREview is a set of requirements which should be:

- “probably” complete
- mutually consistent
- compliant with and providing satisfaction of concerns

The vehicle for the discovery of requirements is the viewpoint and viewpoint analysis is driven by steps designed to ensure their compliance with concerns.

The remainder of this paper uses examples drawn from the railway domain; the specification of an on-board train protection system called "GAAP". The train is controlled by a driver. GAAP’s job is to ensure that the driver respects the operational rules in force on GAAP sections (determined from data collected in real time from the train and the track-side equipment), take corrective action when the driver breaks these rules and provide diagnostic information. Essentially, if the driver allows the train to go too fast or to illegally cross a “stopping point” (such as a signal at stop), GAAP will cause the emergency brakes to be applied.

GAAP must be integrated with an existing execution environment and other on-board systems. A module called "Common on-board software" (COBS) exists through

which the GAAP software communicates with all the hardware interfaces. This provides an interface of functions permitting (e.g.) emergency braking to be invoked, and data permitting GAAP to poll for (e.g.) train speed, distance to next stopping point, etc.

3.1 The PREview process

The PREview process is illustrated in figure 3. The three shaded boxes at the top map directly onto the three segments or phases of the spiral model. The box at the bottom represents the output from the PREview process and the transition to a detailed requirements specification document. This paper is only concerned with the top three phases. Note that the process is not strictly sequential. For example, requirements information may emerge from the discovery/analysis/negotiation cycle while some viewpoints are incomplete or being resolved.

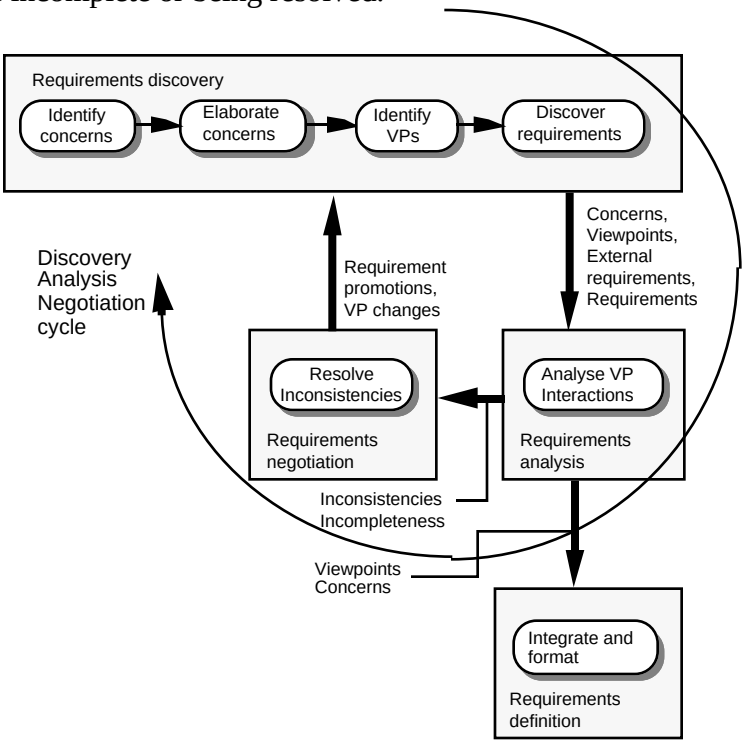

Figure 3 The PREview process

3.2 Concerns

Concern identification and elaboration are the first activities to be performed in the PREview process. PREview makes the assumption that the number of concerns for a particular application will be small; more than 5 or 6 will be difficult to handle and may result in inter-concern conflicts.

Briefly, a concern may be defined as a non-negotiable requirement whose satisfaction is essential to the success of the enterprise. It is “global” in the sense that it has a wide scope in that it potentially affects every aspect of the system rather than, for example, being satisfiable by a single component. If a “concern” does not meet these criteria, it is not a concern.

Figure 4 shows the conceptual relationship between concerns and viewpoints as a “socio-technical pyramid”. At the apex of the pyramid are the viewpoints which

interact directly with the system. At its base are those viewpoints which have the most indirect association with the system, but nevertheless have a stake in it. Viewpoints at each level impose requirements and a viewpoint's level implies nothing about its importance (e.g. a marketing viewpoint may carry more weight than that of an operator). Concerns, however cut through all of these in a vertical axis.

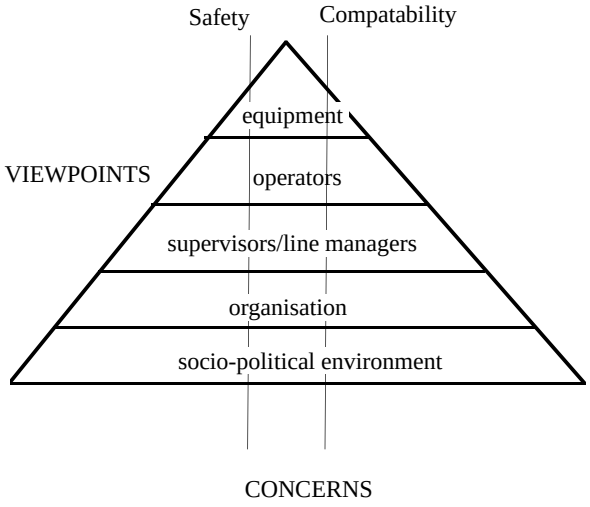

Figure 4 The orthogonality of viewpoints and concerns

Additional characteristics of concerns may include:

- They are typically non-functional. A concern's satisfaction may necessitate the formulation of a number of functional requirements but they will also act to constrain other requirements.

- They are often vaguely expressed, hard to quantify, and hard to validate. For example a safety concern may be expressible at a lower level as an amalgam of requirements expressing availability (e.g. mean time between failure of X time units) and qualitative statements such as "the failure of no single component shall result in a catastrophic loss" but cannot be simply refined as a set of requirements which if met, will guarantee safety to the level desired.

- They are often an intrinsic feature of the domain; safety for transportation systems, security for government information systems, etc.

Concerns must be identified at the outset of a project by discussion with the principal stakeholders; typically the client and developer. This is the first activity in requirements discovery (figure 3). In the GAAP application, two concerns apply: *Safety* and *Compatibility*. Safety is a concern because although GAAP does not control the train it contributes to train safety and because safety enhancement is exploitable for certification and marketing purposes. Compatibility is a concern because GAAP must be integrated with the existing systems' execution cycle and because the COBS module provides the interface through which all communication with other software and hardware modules is made.

Once identified, concerns must be elaborated (figure 3) into a form which is directly applicable. This elaboration takes two orthogonal forms:

- A set of "external requirements".

- A set of questions which compose a check-list.

Concerns are the "conscience" of the requirements engineer who should apply the questions as a test for compliance when a viewpoint's requirements are first discovered. They therefore serve as a first defence against non-compliant requirements. They are applied on the discovery of viewpoint requirements (the 4th activity within requirements discovery in figure 3). For example, a single concern question is derived for Compatibility:

Q1: *Is the requirement consistent with the interface provided by the COBS module?*

Consider the case where a stakeholder articulates a requirement to calculate a minimum braking distance to a degree of accuracy requiring parameters giving: train speed, mass, line gradient and track surface conditions. Applying the concern question to this requirements would prompt checking that these parameter values were indeed available through the COBS interface. Track surface conditions are not monitored via the COBS module so the requirement would be shown to be infeasible.

Of course, sufficient information will not always be available to demonstrate compatibility at the discovery stage and incompatible requirements will inevitably escape detection. However, external requirements provide a second line of defence. Once a viewpoints' requirements have been discovered and documented, they must be validated against the external requirements to ensure their compliance. This is performed within the requirements analysis phase (figure 3). Note that external requirements are not simply constraints. They also represent the initial step in the transformation of concerns into concrete requirements which must be explicitly addressed by the design.

For example, the external requirements derived for the Compatibility concern are:

ER4: *The GAAP software shall be executed in the Modula 2 safety environment of the coded processor.*
ER5: *The GAAP software shall execute within the application cycle of the existing on-board software.*
ER6: *The reaction time of the GAAP software to the change of state of one bit in the variants table shall be 312ms.*
ER7: *The real-time performance of the existing on-board software shall be maintained.*

Clearly, concern elaboration is a complex task which must be performed carefully. However, by investing resources at this very early stage, the elaborated concerns will serve to guard against incompatibilities introduced later. In addition, some classes of concern may employ other techniques to assist with their elaboration. For example, a safety concern may exploit hazard analysis techniques to identify the specific hazards against which the system must provide a defence. Here, a specific external requirement and concern question might be developed for each hazard.

For example, in the GAAP Safety concern, the specific hazards against which GAAP must protect are: excess speed and overshooting a stopping point. These can be elaborated into the following 3 external requirements:

ER1: *The system shall detect the occurrence of excess speed*
ER2: *The system shall detect the occurrence of overshoot*

ER3: *The system shall apply emergency braking when either excess speed or overshoot are detected.*

3.3 Viewpoints

A PREview viewpoint [Sommerville 96] represents a perspective used to map requirements derived from the problem domain onto the system to be developed. A key feature of a viewpoint is the focus which it takes. This defines the scope of the viewpoint's requirements as a function of the problem domain and the components influenced in the system (figure 5). In a typical application, focus will be initially defined in terms of the domain but as analysis proceeds and a system architecture emerges, focus will become "concretised" and more specific. In conventional systems requirements terms, focus may be thought of to evolve into the union of the set of partitions to which a viewpoint's requirements are allocated.

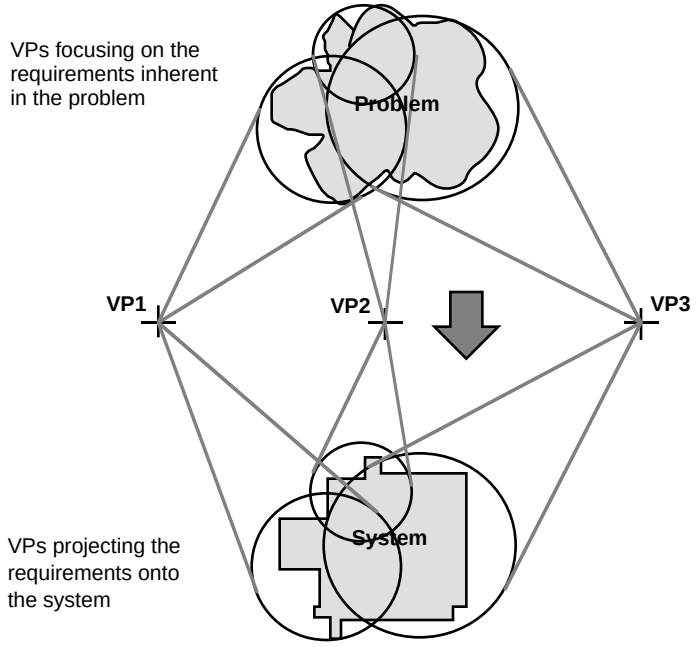

Figure 5 Viewpoints on the problem and the system

Above, we identified viewpoint identification (figure 3) as a key difficulty. PREview recognises this as a problem and suggests the use of both top-down and bottom-up approaches to viewpoint identification. Some viewpoints will be immediately apparent from the domain. Examples of viewpoints identifiable in this bottom-up manner for GAAP include those of Driver, COBS and Safe state assurance. Other viewpoints will only emerge from analysis of the domain and to assist with this, PREview provides guidance for the hierarchical decomposition of the domain to help focus the requirements engineer's search to uncover the viewpoints. To further assist with this, as a starting point we suggest that viewpoints are considered to fall into one of three broad classes:

- Interactors: This includes human operators and interfaced modules/components. These viewpoints represent those at or near the apex of the socio-technical pyramid (figure 4).

- Stakeholders: This is a very broad class which represents viewpoints nearer the base of the socio-technical pyramid (figure 4). Note that we find it convenient to

regard users (conventionally, the archetypal “stakeholders”) as interactors rather than stakeholders. This need not preclude their being regarded as stakeholders in other applications, however.

- Domain phenomena: Such requirements frequently remain implicit. We believe that it is frequently appropriate to make them explicit in order to guard against their being overlooked by developers lacking domain expertise. The GAAP application, for example, includes a Braking characteristics viewpoint. The requirements arising from the physics of train braking systems cannot be attributed to a particular interactor or stakeholder. The domain phenomena classification permits the requirements to be integrated cleanly with other more readily attributable requirements.

In the GAAP application, several stakeholders may be identified including the train operating company, the manufacturer/developer, and the regulatory authority. These can all be further decomposed into particular roles within each of these three organisational stakeholders. This decomposition should continue until the viewpoints derived represent a single cohesive perspective which can be succinctly described as the viewpoint’s focus, and where the sources of the requirements on this focus are identifiable. Note that viewpoints’ foci need not be disjoint.

A PREview viewpoint consists of:

- The viewpoint *name*.
- The viewpoint *focus*. The perspective taken by the viewpoint.
- The *concerns* applicable to the viewpoint. By default, this is the set of all system concerns. Concerns may be eliminated from the viewpoint if it can be demonstrated that they have no constraining effect on the viewpoint.
- The viewpoint *sources*. These explicitly identify the sources of the requirements associated with the requirement. They may be individuals, roles, groups, or documents.
- The viewpoint *requirements*. This is the set of requirements discovered by analysing the system from the viewpoint’s perspective and by consultation with the sources.
- The viewpoint *history*. This records changes to the information recorded in the viewpoint over time. For example, a rationale for why a particular concern need not be considered by the viewpoint should appear here.

Two examples of viewpoints on GAAP are given. Safe state assurance is the first example and represents requirements for maintaining the train in a safe state. It was one of the viewpoints apparent from a bottom up analysis of the domain and is clearly closely related to the safety concern. It is frequently the case that viewpoints emerge which address the similar issues as concerns. This apparent redundancy is not problematic, especially when, as here, the viewpoint has a fairly narrow focus.

Both concerns are considered to constrain the viewpoint: Safety because incorrect requirements may compromise safety; and Compatibility because performance and interface issues are affected by the viewpoint’s requirements.

Analysis of the viewpoint has discovered three requirements: SS1, SS2 and SS3. These are documented in detail elsewhere. It is of course essential to apply version management to all the artefacts of the PREview process such as viewpoints, requirements and concerns.

Name	Safe state assurance
Focus	Detection of dangerous conditions and application of emergency braking
Concerns	Safety Compatibility
Source	Marketing Group C Systems design group A Functional specification of existing protection system software (ref XYY)
Requirements	• SS1 (Detection of excess speed) • SS2 (Detection of overshooting) • SS3 (Frequency of invocation)
Change history	

The second example is the Standards viewpoint. This represents "process" requirements on GAAP's development. It specifies the quality standards to be applied by the development organisation and is thus a stakeholder's viewpoint.

Neither concern is considered to constrain the viewpoint. This removes the obligation to explicitly check the requirements against concern questions and external requirements. However, an explanation of why the concerns are not considered to apply must be recorded as part of the viewpoint's change history.

Name	Standards
Focus	Quality and development standards
Concerns	~~Safety~~ ~~Compatibility~~
Source	Software Quality Plan
Requirements	• S1 (Use of B method) • S2 (Application of ISO 9000)
Change history	1. Compatibility concern inapplicable to this viewpoint as neither standard has any effect on compatability with the existing software. 2. Safety concern inapplicable to this viewpoint as neither standard has any effect on the assurance of safety.

As the examples suggest, we take a broad view of what constitutes a viewpoint. PREview's model can accommodate data sinks/sources, individuals, roles, etc. By recognising domain phenomena, it can also accommodate viewpoints which are not attributable to a concrete entity but which nevertheless contribute fundamental requirements information.

Once a viewpoint has been identified, the requirements arising from that viewpoint can be discovered from the viewpoint's source(s). It is at this point, where the sources are yielding requirements, that concern questions elaborated from the concerns applicable to the viewpoint, should be applied.

3.4 Assurance of compatibility

When the set of identified viewpoints have had their requirements documented, they should be analysed to detect inconsistencies. This is the requirements analysis activity

in PREview. PREview's approach to this is more light-weight than that taken by other methods. PREview is explicitly aimed at the early stages of the requirements process so we do not prescribe particular viewpoints notations. Indeed, requirements for an application such as GAAP will typically be documented by a mixture of natural language, formal, semi-formal and graphical notations. In addition, the requirements will be at a variety of levels of detail and a mixture of functional and non-functional, product and process requirements. Hence, a systematic approach to inconsistency detection is desirable but any automated detection or formal reasoning is infeasible.

We employ a tabular technique based loosely on the "house of quality" used by Quality Function Deployment [Errikson 93]. Here, we tabulate requirements against each other, and where they intersect, perform an assessment of whether they are mutually reinforcing ("+"), independent - i.e. have a neutral effect on each other ("0"), or conflicting ("-"). Notice that the we are not providing a method to perform this assessment, we are merely focusing attention on those requirements which should be assessed. The technique is used both to check for inter-viewpoint conflicts and to check viewpoint requirements' compliance with external requirements.

For example , consider the three (grossly simplified) requirements belonging to the Safe state assurance viewpoint:

SS1 *(Detection of excess speed)*: *If the speed of the train is excessive, emergency braking shall be applied.*
SS2 *(Detection of overshooting)*: *If the front of the train has passed a stopping point, emergency braking shall be applied.*
SS3 *(Frequency of invocation)*: *Detection of excess speed, detection of overshooting and determining the necessity of emergency brake application shall be performed once every iteration of the on-board software application cycle.*

The table below plots these against the external requirements from both the Safety and Compatibility concerns:

		Safety		Compatibility				
		ER1	ER2	ER3	ER4	ER5	ER6	ER7
Safe	SS1	+	0	+	0	0	0	0
state	SS2	0	+	+	0	0	0	0
assurance	SS3	0	0	0	0	-	-	-

Note that SS1 reinforces external requirements ER1 and ER3 - these are all concerned with detection and correction of excess speed. A similar situation exists for SS2.
Note also, that SS3 has a potential conflict with ER5 - 7:

ER5: *The GAAP software must execute within the application cycle of the existing on-board software.*
ER6: *The reaction time of the GAAP software to the change of state of one bit in the variants table must be 312ms.*
ER7: *The real-time performance of the existing on-board software must be maintained.*

The potential conflict arises because of the need to integrate GAAP into the existing on-board application cycle and the consequent need to meet performance constraints. This raises the question of whether requirement SS3 is feasible under these constraints.

The result of the analysis is that all 3 of the Emergency braking viewpoint's requirements will go forward to requirements negotiation (see figure 3): SS1 and SS2 to see if they are redundant and could be simplified; and SS3 to investigate whether it needs to be modified or if a further constraint needs to added. Hence we have detected potential conflicts with the compatibility concern and raised the need for further analysis.

As indicated above, the same technique can be employed to focus the analysis on detecting inconsistencies between different viewpoints' requirements. Obviously, explicitly analysing and checking each requirement against every other requirement quickly becomes infeasible as the number of requirements discovered increases. Instead we recommend that viewpoints' foci are used as a guide to whether two viewpoints' requirements are likely to interact. If their foci intersect (i.e. impose requirements which influence the same components of the system or its environment), then they should be tabulated and checked for consistency.

The sets of conflicting, redundant and non-compliant requirements which form the input to the requirements negotiation phase must, after recommendations have been made for resolving the problems, be fed back into the process at the requirements discovery phase in order to ensure that any changes are themselves compliant with concerns and mutually consistent. An application of PREview will typically result in several iterations of the discovery/analysis/negotiation cycle.

PREview focuses on the early stages of the requirements process so there is still much work to do when the viewpoints, concerns and associated requirements emerge from the discovery/analysis/negotiation cycle. For example, the requirements may be mapped into a structured, object-oriented, or other requirements modelling method and a requirements specification document developed. However, the requirements information which does emerge from PREview will exhibit a high degree of completeness and consistency and, crucially, will embody the principal stakeholder's concerns.

4. Conclusions and further work

PREview is a pragmatic adaptation of an old idea: the use of viewpoints for requirements engineering. However, its very pragmatism is one of its principal features. The need for this was established by our industrial collaborators at the outset of the REAIMS project and has been the method's underpinning philosophy. It has been designed to be adaptable to an organisation's existing requirements process. It provides a mechanism to make mature organisations' good (but informally applied) practices more systematic and applicable by less mature organisations. As such it is designed to be adopted in an evolutionary way rather than requiring a revolutionary change to a development organisation's existing practice. PREview is not prescriptive about notations or methods so it can be integrated with an existing requirements method as a front-end process.

In addition to the three stated benefits of the use of viewpoints:

- Completeness
- Separation of concerns
- Traceability

PREview adds the explicit recognition of the importance or organisational needs and priorities - the concerns of the principal stakeholders. By integrating these within the viewpoint model, the risk of their remaining incompletely articulated and of recognition of their crucial importance becoming diluted is reduced.

PREview is currently being evaluated by our industrial partners. The GAAP example used in section 3 has been derived by reverse engineering a completed project. It has also been adapted to a requirements reuse process based on accumulating experience of in-service systems to be reused as requirements in future products.

One of the results of these applications has been the discovery that although designed to be adapted to an organisation's existing process, the organisation's process must be defined in order to make adoption feasible. Future work will include an investigation of how PREview can be adapted for adoption by organisations with less mature processes. We are also developing tool support based on low-cost spreadsheet and database software which we hope will enhance management of the requirements information. An additional aspect of PREview which we have not touched upon is the use of viewpoints for process analysis. This is described in [Sommerville 95].

In summary, PREview helps improve the quality of requirements specification by providing a framework for the early phases of the requirements process and enshrines the need to comply with and to satisfy the overall, binding concerns defining the success or failure of a project. Failure to achieve this inevitably leads to expensive rework or project failure. In some respects, it is "less advanced" than other approaches. We do not propose technological solutions such as automated conflict analysis nor have we invented expressive notations or new processes. However, we have built on previous research to develop an evolutionary approach which addresses outstanding requirements engineering problems in a way which is pragmatic and relevant to industrial needs.

5. Acknowledgements

We gratefully acknowledge the contributions made to this work by our collaborators on the REAIMS project: Adelard, Aerospaciale Aeronautique, Aerospaciale Protection Systemes, Digilog, GEC Alsthom Transport, RWTÜV and The University of Manchester.

6. References

Boehm, B. 1988. A Spiral Model of Software Development and Enhancement. *IEEE Computer*, **21** (5)

Boehm, B., Bose, P., Horowitz, E., Lee, M-J. 1994. Software Requirements as Negotiated Win Conditions. *1st International Conference on Requirements Engineering*
(ICRE), Colorado Springs. Co

Booch, G. 1994. *Object-oriented Analysis and Design with Applications,*. Benjamin Cummings, Redwood City, Ca.

Davis, A. 1993. Software Requirements: Objects, Functions and States. Prentice-Hall, London.

Easterbrook, S. 1993. Domain Modelling of Hierarchies of Alternative Viewpoints. *Proc. 1st IEEE International Symposium on Requirements Engineering,* San Diego, Ca.

Easterbrook, S. and Nuseibeh, B. 1996. Using ViewPoints for inconsistency management. *BCS/IEE Software Eng. J.* , **11**(1)

Errikson, I. and McFadden, F. 1993. Quality Function Deployment: A Tool to Improve Software Quality. *Information and Software Technology* , **35**(9)

Finkelstein, A., Sommerville, I. 1996. The Viewpoints FAQ. Editorial. *BCS/IEE Software Eng. J.* , **11**(1)

Hutchings, A., Knox, S. 1995. Creating Products Customers Demand. *Comm. of the ACM*. **38** (5).

Jacobson, I., Christensen, M., Jonsson, P. and Overgaard, G. 1993. *Object-Oriented Software Engineering.* Addison-Wesley, Wokingham.

Jackson, M. A. 1983. *System Development*. Prentice-Hall, London.

Jackson, D. and Jackson, M. 1996. Problem decomposition for reuse. *BCS/IEE Software Eng. J.* , **11**(1)

Kotonya, G. and Sommerville, I. 1996. Requirements engineering with viewpoints. *BCS/IEE Software Eng. J.* , **11**(1)

Leite, J. C. S. P. and Freeman, P. A. 1991. Requirements Validation through Viewpoint Resolution. *IEEE Trans. on Software Eng.* , **17**(12)

Mullery, G. 1979. CORE - A Method for Controlled Requirements Specification. In *Proc. 4th Int. Conf. on Software Engineering,* Munich.

Nuseibeh, B., Kramer, J. and Finkelstein, A. 1994. A Framework for Expressing the Relationships between Multiple Views in Requirements Specifications. *IEEE Trans. on Software Eng.* , **20**(10)

Ross, D. T. 1977. Structured Analysis (SA). A Language for Communicating Ideas. *IEEE Trans. on Software Engineering.* , **SE-3**(1)

Rumbaugh, J., Blaha, M., Premerlani, W., Eddy, F. and Lorensen, W. 1991. *Object-oriented Modeling and Design.* Prentice-Hall, London.

Schoman, K. and Ross, D. T. 1977. Structured Analysis for Requirements Definition. *IEEE Trans. on Software Engineering.* , **SE-3**(1)

Sommerville, I., Sawyer, P., Kotonya, G. and Viller, S. 1995. Process Viewpoints. In *Proc. Proc. 5th European Workshop on Software Process Technology,* Amsterdam, The Netherlands.

Sommerville, I., Sawyer, P. 1996. Viewpoints: Principles, Problems and a Practical Approach to Requirements Engineering. Submitted for publication in: *Annals of Software Engineering.*

Zave, P. 1995. Classification of Research Efforts in Requirements Engineering. In *Proc. 2nd IEEE International Symposium on Requirements Engineering,* York, England.